XRPLA benchmark report

October 2026 audit: correctness against xrpld and Clio, restore and storage, and query speed

Every number in this report is read from the raw results in `docs/benchmarks/2026-10-audit/`, which NOTES.md describes file by file. This PDF is rebuilt from them by `report/make_report.py`.

Summary

- **Correctness.** Against stock xrpld 3.4.1, XRPLA returned the same data in every comparison: 120,617 answers in the parity campaign across 108 intervals of history, 14,111 on the restored 107M range (its 3 differences were ping's role field, from an admin connection), 53,665 on four older stretches (2015-2024), and 426,171 account_tx delegate pages.
- **Clio.** A Clio 2.8 restored from XRPLA's publication answered 14,092 requests exactly as a Clio whose database Clio's own ETL built (the only differences, 6, were the nodes' version numbers).
- **Storage.** For the same 700,000 ledgers, XRPLA's piece cache is 23 GB, against 182 GB for Clio's Cassandra and 711 GB for xrpld's NuDB and SQLite.
- **Restores.** XRPLA restored those ledgers into a working xrpld in 14 min 19 s with downloads cached (15 min 46 s downloading from R2), and 100,000 older ledgers in 25 s to 1 min 16 s. A Clio restore loaded 951,674,801 rows in 1 h 35 min.
- **Point reads.** XRPLA leads: 71,521 requests/s over WebSocket against 40,410 for xrpld and 19,430 for Clio, on 7.0 busy cores against 8.7 and 14.5.
- **Historical owner and history reads.** xrpld and Clio lead: on the full mix XRPLA serves 1,548 requests/s with a 32 GB cache and the object-read change, against 5,603 for Clio and 5,333 for xrpld. The audit raised XRPLA from 422 with the old 16 GB default; the remaining gap is memory per decoded segment (section 5).

1. Setup

Hetzner (publisher-host; Xeon Gold 5412U, 48 threads, 126 GB RAM, NVMe RAID). Servers run on cores 16-39 and the load generator (perf.py: 64 requests in flight, 4 processes, closed loop) on cores 40-47, beside the production publisher, a soak test and a Postgres replica. All three servers hold ledgers 107,000,001-107,697,250:

- **XRPLA** (A′): a piece cache of the range, synced from the R2 publication, served by `xrpla serve`.
- **xrpld 3.4.1** (X): NuDB and SQLite restored from the publication by `xrpla restore --to nudb`, running on mainnet from the restored tip.
- **Clio 2.8.0** (C) on Apache Cassandra 4.1.10 (G1, 24 GB heap), loaded by `xrpla restore --to clio`, following the tip through its ETL.

Request sets. The *point* mix is account_info, tx, ledger, ledger_entry and server_info. The *full* mix adds account_lines, account_objects, account_tx and book_offers, which read many objects or a long history at an older ledger. Requests come from a corpus of recent and older ledgers of the range.

Test server (test-host, xrpla-test.example; Ryzen 7 7700, 16 threads, 61 GB RAM): XRPLA's open full-history node, ledgers 32,570 to the tip, with a 32 GB decode cache.

2. Correctness

Each comparison derives requests from each sampled ledger's contents (accounts, books, tokens, transactions), asks both servers, and compares the answers whole, after removing only fields that describe the server. XRPLA's answers were also asked over its WebSocket and compared with its HTTP answers.

Comparison	Reference	Answers	Different
Parity campaign: 108 intervals across all history	xrpld (s1/s2, xrplcluster arbiter)	120,617	0
same requests over WS, HTTPS and WSS	XRPLA over HTTP	361,851	0
Restored 107M range, 3 seeds of 300 ledgers	xrpld 3.4.1 (local)	14,111	3 (ping role)
Older history: 4 stretches of 100,000 ledgers	xrpld 3.4.1 (local)	53,665	0
account_tx delegate filter, every page	xrpld 3.4.1	426,171	0
Clio restored from XRPLA, 3 seeds of 300 ledgers	Clio node (own ETL)	14,092	6 (version)
Invariants: derived answers against verified state	XRPLA's own state	14,652	0

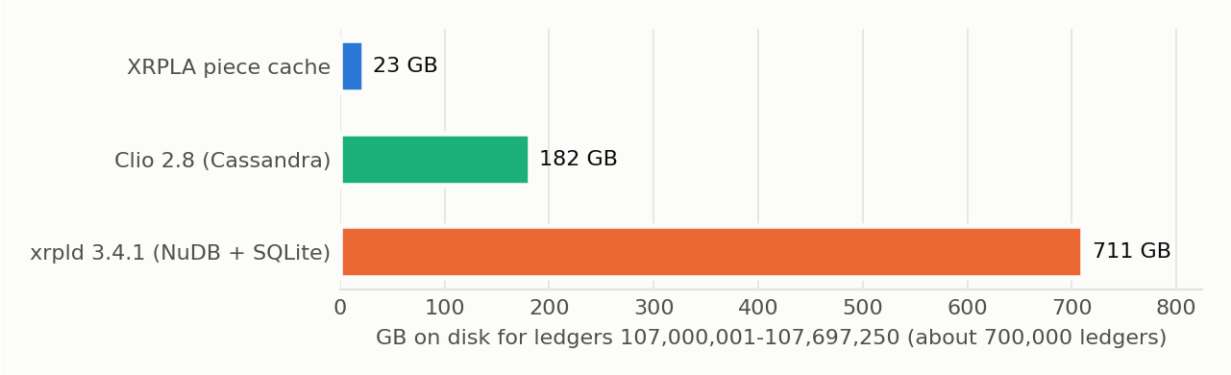
The campaign's 38 timeouts and 24 reference problems were on the public reference servers, which were not compared. The 3 differences on the 107M range, one per seed, are `ping`'s `role` field, which xrpld adds for an admin connection. The invariant checker (tools/xrpla_invariants.py) needs no reference: it checks account_tx, tx, NFT, MPT and order-book answers against the hash-verified state and transactions of 150 sampled ledgers; 88 checks whose listings ran past its page cap (accounts holding over 160,000 NFTs) were skipped, not passed.

Older history

Stretch	Restore	xrpld database	Peak memory	Piece cache sync	Answers	Different
14,000,001-14,100,000 (2015)	25 s	18 GB	4.3 GB	3 min 16 s	13,707	0
38,000,001-38,100,000 (2018)	51 s	44 GB	11.6 GB	7 min 47 s	13,234	0
63,000,001-63,100,000 (2021)	57 s	55 GB	11.4 GB	11 min 28 s	12,921	0
88,000,001-88,100,000 (2024)	1 min 16 s	48 GB	13.8 GB	37 min 52 s	13,803	0

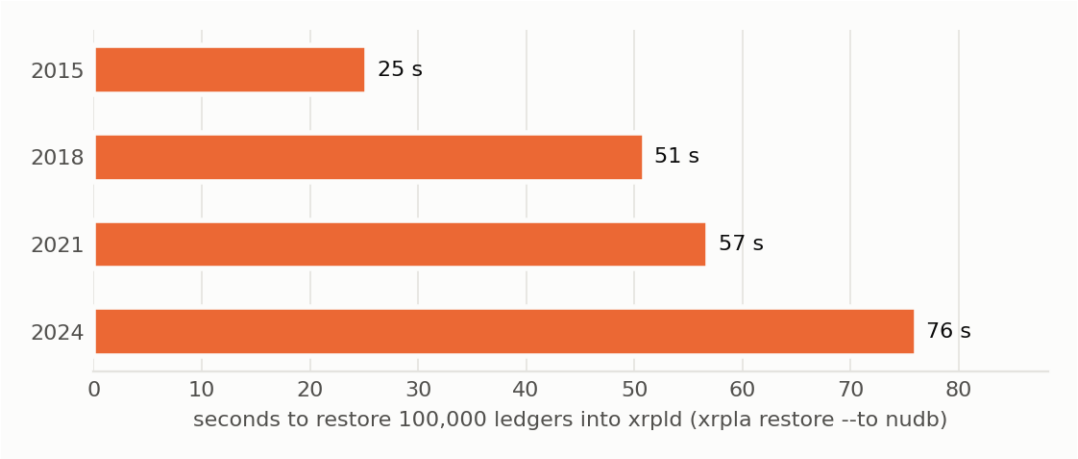
Each stretch was restored into xrpld's NuDB and SQLite by xrpla restore (543c523), served by xrpld 3.4.1 standalone, and compared with XRPLA serving a piece cache of the same range. A standalone xrpld counts only its loaded ledger as present, so it was started with --force_ledger_present_range over the stretch, and on public ports (an admin connection has no minimum limits).

3. Storage and restore



System (ledgers 107,000,001-107,697,250)	On disk	Built by	Time
XRPLA piece cache	23 GB	xrpla sync (from R2)	17 min 06 s
Clio 2.8 on Cassandra	182 GB	xrpla restore --to clio	1 h 35 min (load)
xrpld 3.4.1, NuDB + SQLite	711GB	xrpla restore --to nudb	14 min 19 s (cached), 15 min 46 s

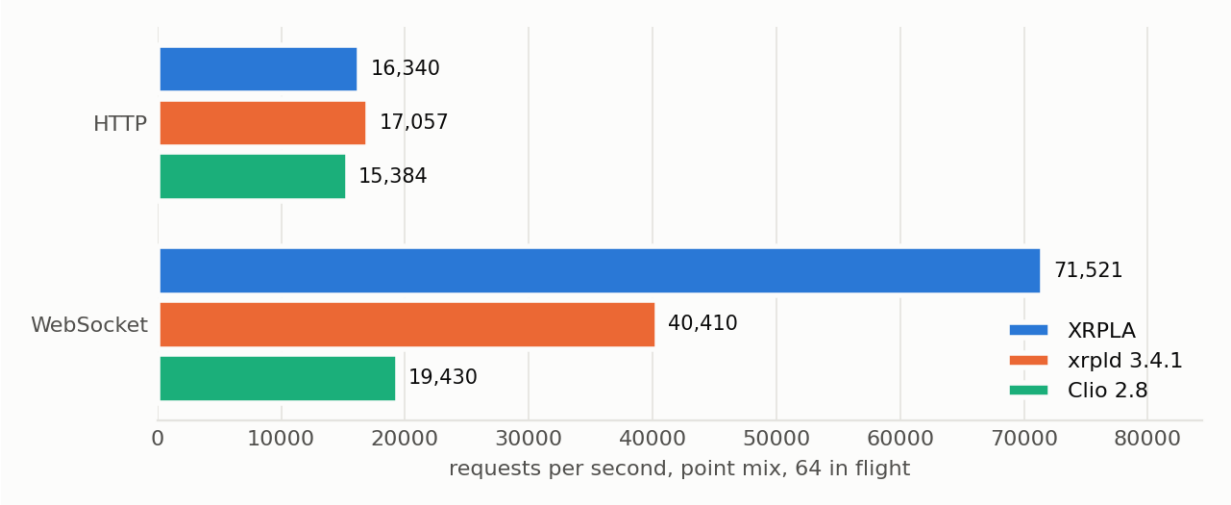
The first xrpld restore (15 min 46 s, downloading from R2) lost NuDB spill records to a bug in its final checkpoint; it was fixed in cb78f55 and the restore run again (14 min 19 s, downloads cached), and that database is the one compared. Every restore checks each ledger's state root and its parent chain before writing it. The xrpld restore then started on mainnet from the restored tip, read its state from NuDB rather than from peers, and reached *full*. The Clio restore's plan was built in 57 minutes, against more than 4 h 43 min for the code before the audit (whose run was stopped); the load of all 951,674,801 rows then took 1 h 35 min, and Clio continued from the restored tip.



Restoring older history is fast because state was smaller: 18 GB for 100,000 ledgers of 2015 against 48-55 GB for 2021 and 2024.

4. Query speed

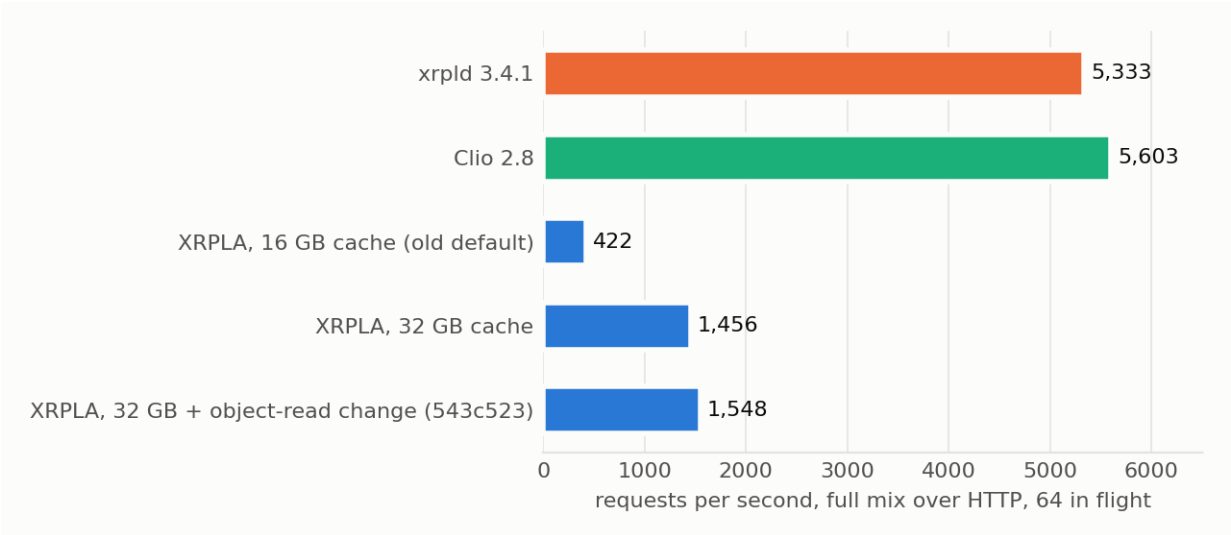
Point reads



Point mix (requests/s)	XRPLA	xrpld 3.4.1	Clio 2.8
HTTP	16,340	17,057	15,384
WebSocket	71,521	40,410	19,430

Over HTTP every request opens a new connection, which bounds all three servers alike. Over WebSocket XRPLA answers about 1.8× as many as xrpld and 3.7× as many as Clio.

Historical owner and history reads

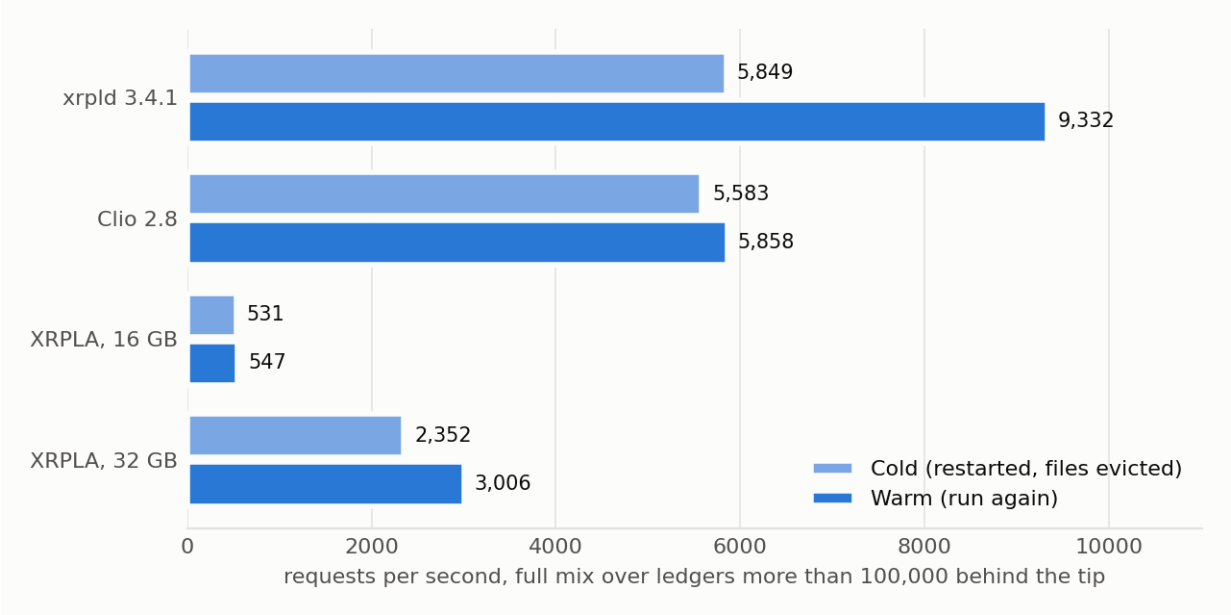


Full mix	Requests/s	Server cores busy
xrpld 3.4.1	5,333 (runs 3,795-6,871)	4.4
Clio 2.8 (Clio and Cassandra together)	5,603	13.2
XRPLA, 16 GB cache (the old default on this machine)	422	
XRPLA, 32 GB cache	1,456	19.3
XRPLA, 32 GB cache, object reads skip the transaction section	1,548	

Median latency, full mix (ms)	xrpld	Clio	XRPLA 16 GB	XRPLA 32 GB
account_lines	10.6	9.0	395.0	107.2
account_objects	10.6	10.3	388.9	105.5
account_tx	17.0	8.7	443.3	124.2
book_offers	10.0	88.9	470.6	126.4
account_info	10.1	8.3	0.3	0.3
tx	13.9	2.9	1.3	0.4

The full mix is where XRPLA trails. Its owner and history methods read objects at an older ledger from compressed pieces; each object read decodes the page of the ledger that last changed it, and that needs the segment's dictionary in memory. Section 5 explains what was measured and changed.

Cold reads



Full mix, older ledgers (requests/s)	Cold	Warm
xrpld 3.4.1	5,849	9,332
Clio 2.8	5,583	5,858
XRPLA, 16 GB	531	547
XRPLA, 32 GB	2,352	3,006

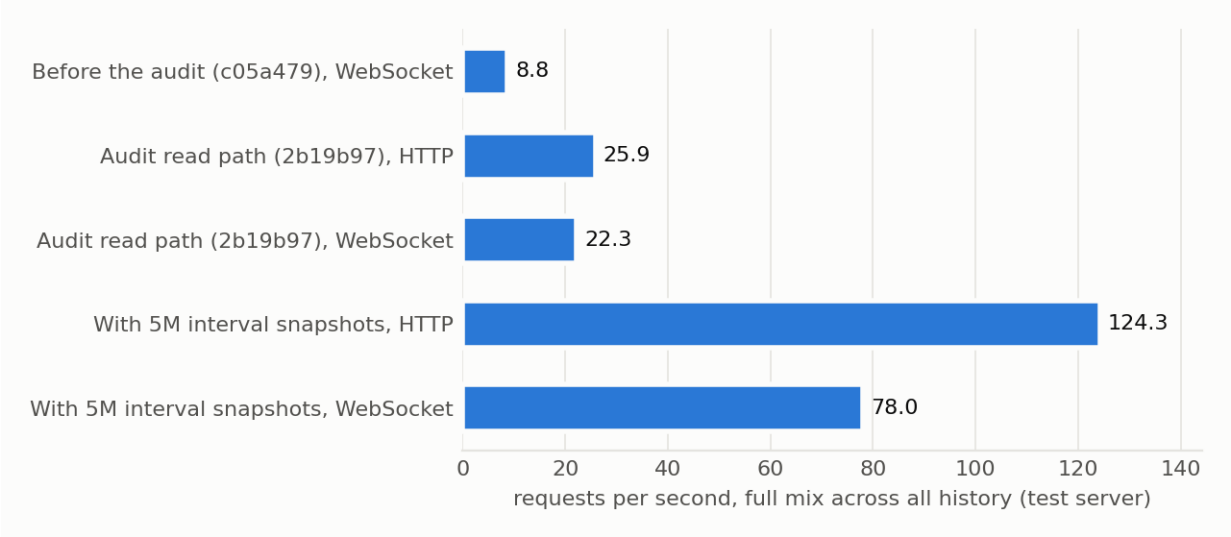
Each server was restarted and its files evicted from the page cache, then asked only about ledgers more than 100,000 behind the range's tip; "warm" is the same run again.

5. The account_lines investigation

Under account_lines alone the full-history node served 4,474 requests/s at a 2.4 ms median with a 32 GB cache, so the warm path is fast. The full mix on Hetzner was slow because the cache was too small for the working set: a busy 700,000-ledger span has about 180 segments, each holding about 100 MB once its dictionary is loaded, and the 16 GB default kept reloading them.

- **Decode cache budget:** 32 GB raised the full mix from 422 to 1,456 requests/s and cold reads from 531 to 2,352. The default is now a quarter of memory, between 3 and 64 GB (1f4a40a).
- **Object reads skip the transaction section** (543c523): 1,452 to 1,548 requests/s at 32 GB.
- **Measured and dropped:** more resume points per segment (no change), keeping decompressed history sections (no change), and a memory-mapped dictionary cache, which doubled throughput on a memory-starved server but stalled on page faults on a real one (about 10 requests/s just after a restart).

Full history on the test server



The test server serves all of history, so a full-mix request can land anywhere in 107 million ledgers. The audit's read path and interval snapshots every 5 million ledgers (each older ledger reads its state from the nearest snapshot) raised it from 8.8 to 78 requests/s over WebSocket (124 over HTTP).

Single request on the test server (seconds)	Cold, before	Cold, audit	Warm, before	Warm, audit
book_offers XRP/USD.Bitstamp @80,000,000	5.265	2.613	5.038	2.432
ledger_data 256 @80,000,000	17.359	2.958	0.020	0.011
account_objects @80,000,000	1.967	0.592	0.050	0.029
account_tx 200 @80,000,000	0.073	0.074	0.039	0.029
ledger transactions+expand @80,000,000	0.552	0.003	0.017	0.003
book_offers XRP/USD.Bitstamp @100,000,000	11.377	5.760	10.847	5.361
ledger_data 256 @100,000,000	9.902	1.777	0.023	0.012
account_objects @100,000,000	0.879	0.268	0.064	0.039
account_tx 200 @100,000,000	0.046	0.039	0.031	0.022
ledger transactions+expand @100,000,000	0.244	0.006	0.015	0.007

6. Caveats

- The xrpld speed runs paired xrpld with XRPLA on the earlier build 2b19b97 at the 16 GB default; the XRPLA rows at 32 GB come from runs on the same machine, corpus and cores, after X's database had been removed. Its point-mix numbers are unchanged across builds.
- Clio followed the network tip through its ETL during its runs, and XRPLA served a fixed range. Clio's CPU and memory include Cassandra's.
- All servers shared Hetzner with production services (the publisher, a soak test, a Postgres replica), pinned to their own cores.
- The test server's full-history numbers come from a single machine with 61 GB of RAM serving 107 million ledgers; they are not comparable with the Hetzner runs over one range.
- Each speed figure is the mean of the runs listed in the raw files (usually two per server, mix and transport, alternating server order).

7. Data

All raw results are in docs/benchmarks/2026-10-audit, described in NOTES.md: the parity campaign (parity-campaign/), the xrpld comparison and speed runs (hetzner-x-*), Clio (clio-comparison/, clio-methods/, clio-restore/), older history (older-history/), the account_lines investigation (account-lines/), the invariant checks (invariants/) and the test server runs (test-server-*).